Lecture 20
SMT and Theorem Proving

Armando Solar-Lezama partially based on slides by
Isil Dillig
http://www.cs.utexas.edu/~isil/cs780-02/lecture16.pdf

What is a theory

A language of expressions and predicates over those expressions where we have a satisfiability procedure for conjunctions over predicates

Candidate Logic

Goals: $G := L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x\,.\ G$

Hypotheses $H := L \mid true \mid H_1 \wedge H_2$

Literals $L := p(E_1, \ldots, E_k)$

Expressions $E := x \mid f(E_1, \ldots, E_m)$

} Background theory
Gives meaning to these

Candidate Logic

Goals: $G := L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x\,.\ G$

Hypotheses $H := L \mid true \mid H_1 \wedge H_2$

Literals $L := p(E_1, \ldots, E_k)$

Expressions $E := n \mid f(E_1, \ldots, E_m)$

Note that our alternative VC gen procedure produces formulas in this form!

- Assuming invariants, preconditions and postcond are all from H.

Meaning of a formula

$\models G$ means that formula G holds

- $\models true$
- $\models G_1 \wedge G_2$ when $\models G_1$ *and* $\models G_2$
- $\models \forall x \,.\, G$ when for all $n \in D$ $\models G[n/x]$ where D is the domain of x
- $\models H \Rightarrow G$ when $\models G$ whenever $\models H$
- $\models p(E_1, \ldots E_k)$ when p is true according to the theory

A simple prove procedure

What we want:

- $prove(G) = true$ iff $\models G$

A simple algorithm

$$prove(G) = prove(true, G)$$

- $prove(H, true) = true$
- $prove\big(H, G_1 \wedge G_2\big) = prove\big(H, G_1\big) \wedge prove(H,\ G_2)$
- $prove\big(H,\ H_1 \Rightarrow G_2\big) = prove(H \wedge H_1, G_2)$
- $prove(H,\ \forall x\,.\ G) = prove(H,\ G\big[a/x\big])$ where a is fresh
- $prove(H, L) = ?$

Theorem prover for literals

Reduces the problem to $prove(H, L)$

- H is a conjunction of literals

$$L_1 \wedge \ldots \wedge L_k \Rightarrow L$$

$$\neg(L_1 \wedge \ldots \wedge L_k \wedge \neg L)$$

$$prove(H, L) = Unsat(H \wedge \neg L)$$

Example: Linear Inequalities

Goals:	$G \coloneqq L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x. G$
Hypotheses	$H \coloneqq L \mid true \mid H_1 \wedge H_2$
Literals	$L \coloneqq p(E_1, \ldots, E_k)$
Expressions	$E \coloneqq n \mid f(E_1, \ldots, E_m)$

Expressions: $c * x_i \mid E + E$

Literals: $E \geq 0 \mid \mathrm{E} = 0$

$$\forall y, y_0, k, k_0, t,\ y',\ t',\ \left(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0\right) \Rightarrow \quad y = k_0 + t$$

$$\wedge \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1)$$

$$\wedge \quad y' = k_0 + t' \wedge \neg(t' > 0) \Rightarrow y' \leq k_0$$

Example: Linear Inequalities

Goals:	$G \coloneqq L \mid true \mid G_1 \wedge G_2 \mid H \Rightarrow G \mid \forall x. G$
Hypotheses	$H \coloneqq L \mid true \mid H_1 \wedge H_2$
Literals	$L \coloneqq p(E_1, \dots, E_k)$
Expressions	$E \coloneqq n \mid f(E_1, \dots, E_m)$

$$not\ (\exists y, y_0, k, k_0, t,\ y',\ t',\ not\big(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0\big) \quad \Rightarrow \quad y = k_0 + t$$

$$\wedge \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1)$$

$$\wedge \quad y' = k_0 + t' \wedge \neg(t' > 0) \Rightarrow y' \leq k_0$$

Linear Inequalities

$$Prove(true, \left((y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) \Rightarrow \begin{array}{c} y = k_0 + t \\ \wedge \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \quad y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array} \right))$$



$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \left(\begin{array}{c} y = k_0 + t \\ \wedge \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge \quad y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array} \right))$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \left(\begin{array}{cc} & y = k_0 + t \\ \wedge & y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge & y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array} \right))$$



$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y = k_0 + t)$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \quad y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1))$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0)$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \left(\begin{array}{cc} & y = k_0 + t \\ \wedge & y' = k_0 + t' \wedge \quad (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge & y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 \end{array} \right))$$



$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, y = k_0 + t)$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge \quad y' = k_0 + t' \wedge \quad (t' > 0),\ y' - 1 = k_0 + (t' - 1))$$

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y' = k_0 + t' \wedge (t' \leq 0),\ y' \leq k_0)$$

Linear Inequalities

$$Prove(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0, \left(\begin{array}{ccc} & & y = k_0 + t \\ \wedge & y' = k_0 + t' \wedge & (t' > 0) \Rightarrow y' - 1 = k_0 + (t' - 1) \\ \wedge & y' = k_0 + t' \wedge (t' \leq 0) \Rightarrow y' \leq k_0 & \end{array} \right))$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y \neq k_0 + t)$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge \quad y' = k_0 + t' \ \wedge \ t' > 0 \wedge y' - 1 \neq k_0 + (t' - 1))$$

$$Unsat(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0 \wedge y' = k_0 + t' \wedge t' \leq 0 \wedge y' > k_0)$$

Is this enough to check VCs?

Given a definition of VC

- $VC(skip,\ \{Q\}) = Q$
- $VC(x = e, \{Q\}) = Q[e/x]$
- $VC\big(C1; C2\{Q\}\big) = VC(C1\{VC(C2\{Q\})\})$
- $VC\big(if\ B\ then\ C1\ else\ C2\{Q\}\big) = \Big(B\ and\ VC\big(C1\{Q\}\big)\Big) or\ (\neg B\ and\ VC\big(C2\{Q\}\big))$
- $VC\big(while_I\ e\ do\ c,\ B\big) =$

$I \wedge \ \forall x_1,\ \ldots x_n\ I \Rightarrow \big(e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow B\big)$

  - Where x_i are variables modified in c.

- $VC\big(assert\ B\ \{Q\}\big) = \{Q \wedge B\}$
- $VC\big(assume\ B\{Q\}\big) = \{B \Rightarrow Q\}$

Is this enough to check VCs?

Given a definition of VC

- $VC(skip,\ \{Q\}) = Q$
- $VC(x = e, \{Q\}) = Q[e/x]$
- $VC\big(C1; C2\{Q\}\big) = VC(C1\{VC(C2\{Q\})\})$
- $VC\big(if\ B\ then\ C1\ else\ C2\{Q\}\big) = \Big(B \Rightarrow VC\big(C1\{Q\}\big)\Big) \wedge (\neg B \Rightarrow\ VC\big(C2\{Q\}\big))$
- $VC\big(while_I\ e\ do\ c,\ B\big) =$

$I \wedge\ \ \forall x_1,\ \ldots x_n\ I \Rightarrow \big(e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow B\big)$

  - Where x_i are variables modified in c.

- $VC\big(assert\ B\ \{Q\}\big) = \{Q \wedge B\}$
- $VC\big(assume\ B\{Q\}\big) = \{B \Rightarrow Q\}$

Equality with uninterpreted functions

Equality with uninterpreted functions

- Symbols: $=$, $\neq$, $f, g, \ldots$
- Axioms:
  - $\dfrac{}{E = E}$ $\qquad$ $\dfrac{E_2 = E_1}{E_1 = E_2}$ $\qquad$ $\dfrac{E_1 = E_2 \qquad E_2 = E_3}{E_1 = E_3}$
  - $\dfrac{E_1 = E_2}{f(E_1) = f(E_2)}$

Equality with UF (Congruence closure algorithm)

x=y

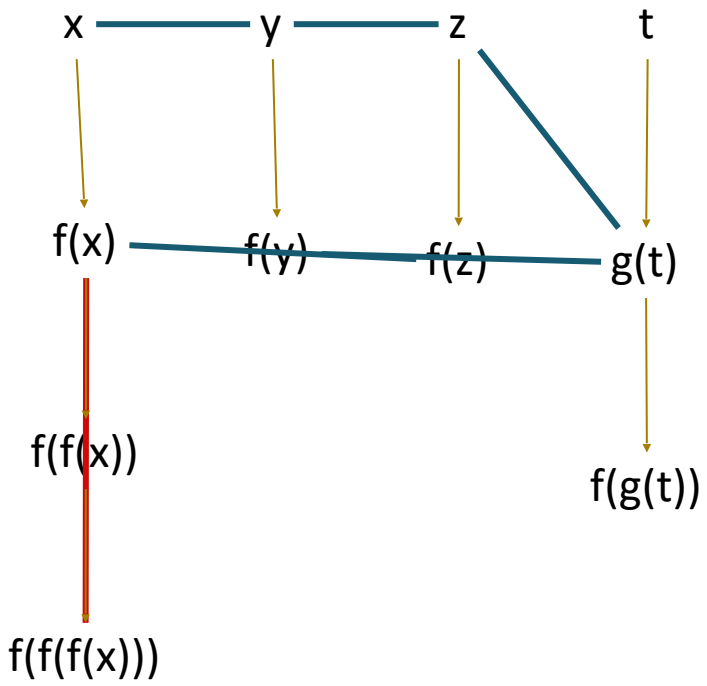
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

f(x) != f(f(f(x)))



Equality with UF

x=y

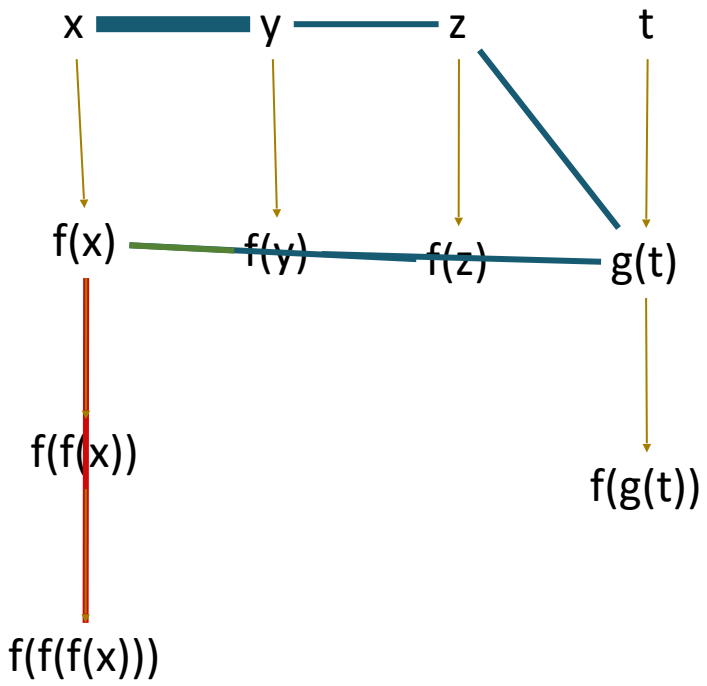
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

f(x) != f(f(f(x)))



Equality with UF

x=y

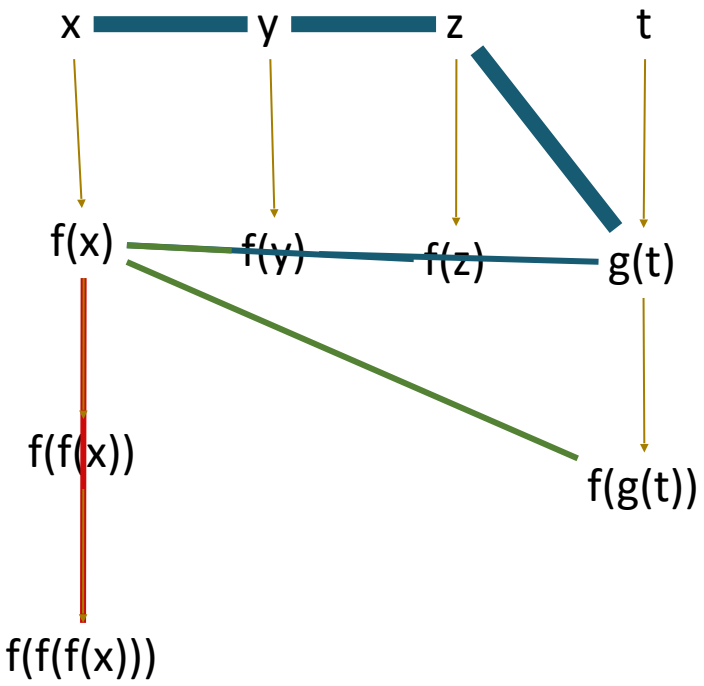
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

f(x) != f(f(f(x)))



Equality with UF

x=y

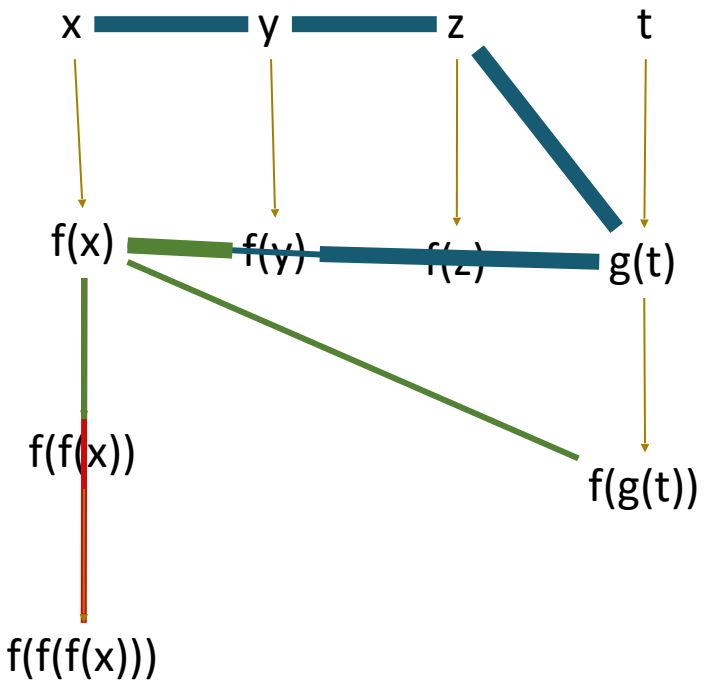
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

f(x) != f(f(f(x)))



Equality with UF

x=y

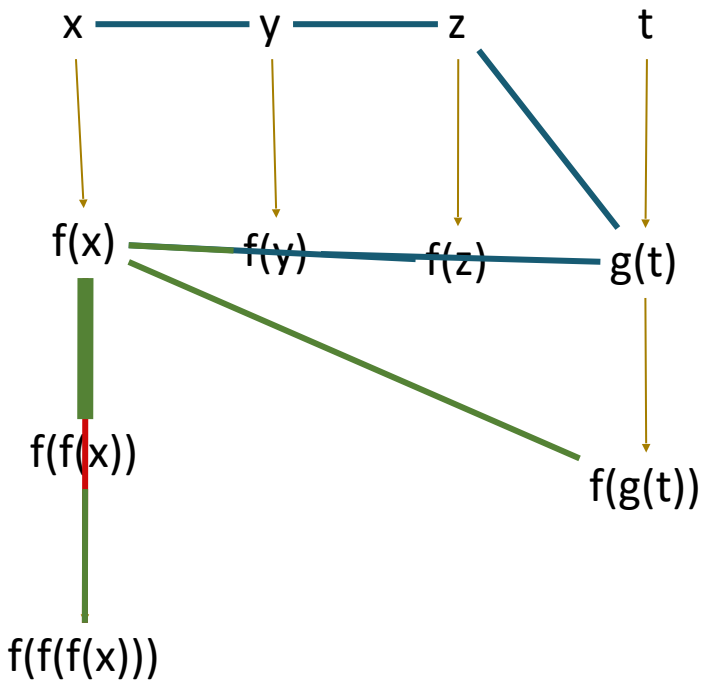
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

f(x) != f(f(f(x)))



Equality with UF

x=y

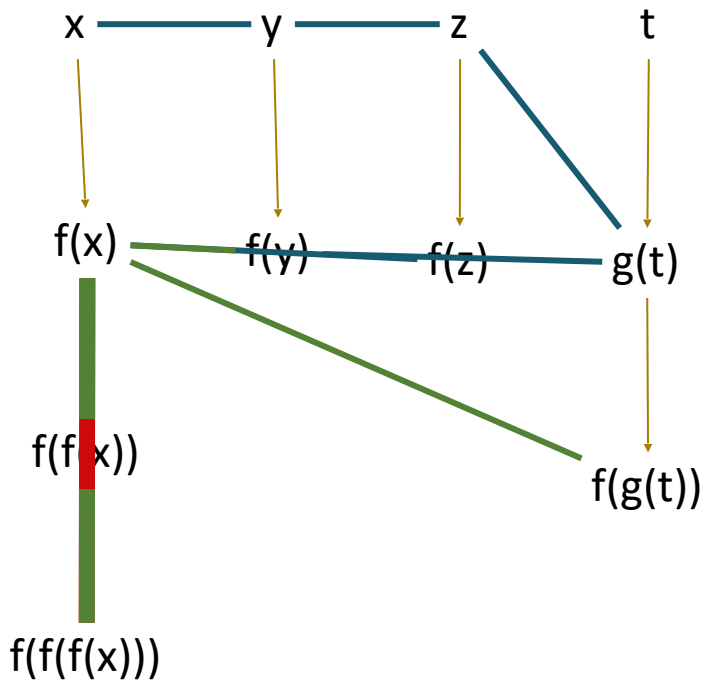
y=z

f(x)=f(z)

g(t)=z

f(y) = g(t)

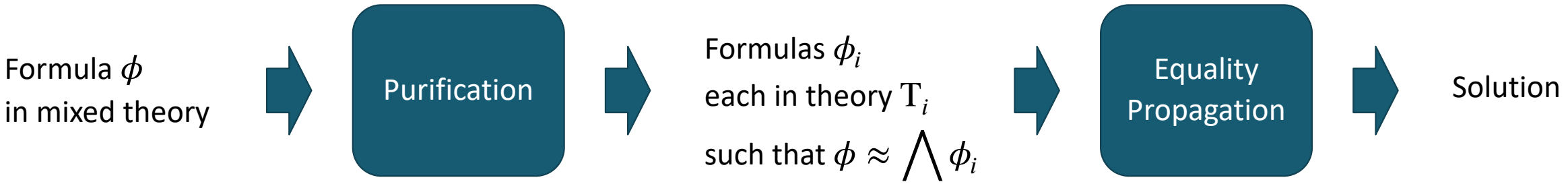
f(x) != f(f(f(x)))



Contradiction!

Nelson-Oppen

Two step process



Purification

Break up formula ϕ into a set of formulas ϕ_i each in theory T_i

such that ϕ and $\bigwedge \phi_i$ are equisatisfiable

Steps

- For every expression $E(f_1, \ldots, f_n)$ if E belongs to some theory T_i and $f_k$belongs to a different theory T_j, replace $E(\ldots f_k \ldots) \rightarrow E(\ldots w \ldots)$ and add a constraint $w = f_k$ to ϕ_j (w is a fresh variable)
- For every literal $p(E_1, \ldots, E_n)$ if p belongs to some theory T_i and $E_k$belongs to a different theory T_j, replace $p(\ldots E_k \ldots) \rightarrow p(\ldots w \ldots)$ and add a constraint $w = E_k$ to ϕ_j (w is a fresh variable)

Example

$$f\big(x+y\big) \neq f\big(g(u)+g(v)\big) \wedge g(u) = y \wedge g(v) = x$$

$f\big(w_1\big) \neq f\big(w_2\big)$
$g(u) = y$
$g(v) = x$
$w_1 = x + y$
$w_2 = w_3 + w_4$
$w_3 = g(u)$
$w_4 = g(v)$

Equality Propagation

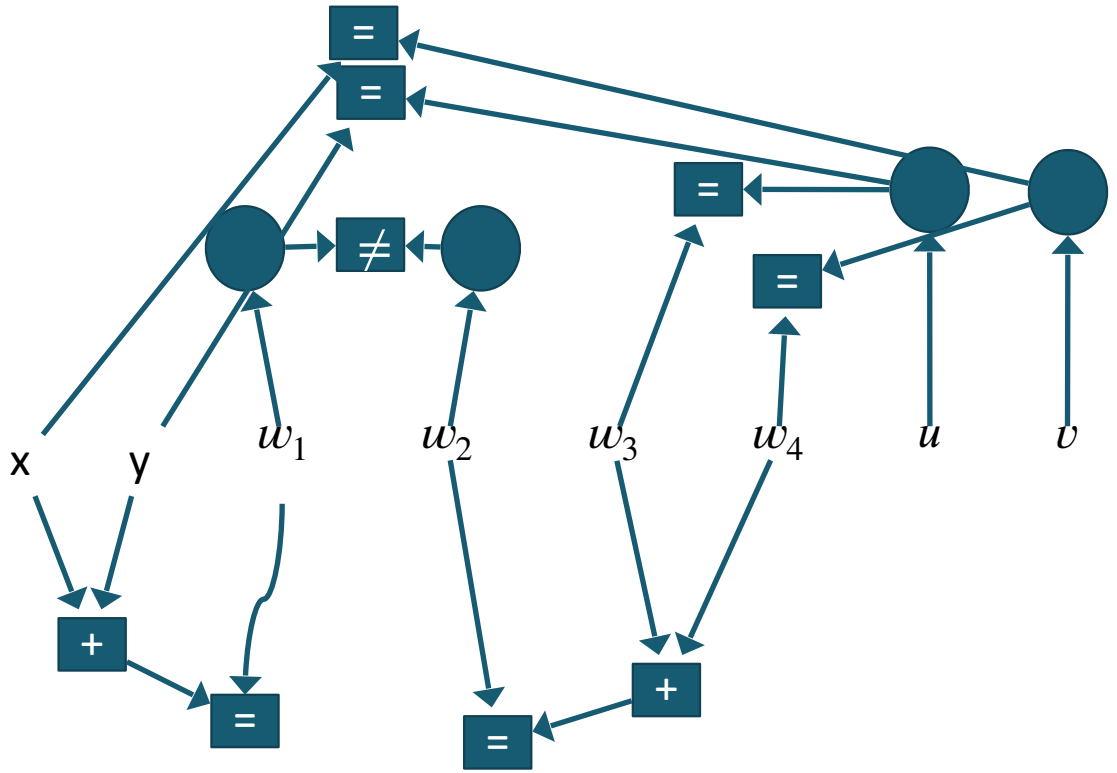
1) Run each UNSAT procedure independently
   - If any returns UNSAT we are done
2) Broadcast any equalities that were discovered
   - If no new equalities were discovered, return SAT

Uninterpreted Fun.

$$f(w_1) \neq f(w_2)$$
$$g(u) = y$$
$$g(v) = x$$
$$w_3 = g(u)$$
$$w_4 = g(v)$$

Linear Integer Arithmetic

$$w_1 = x + y$$
$$w_2 = w_3 + w_4$$



Equality Propagation

1) Run each UNSAT procedure independently
   - If any returns UNSAT we are done
2) Broadcast any equalities that were discovered
   - If no new equalities were discovered, return SAT

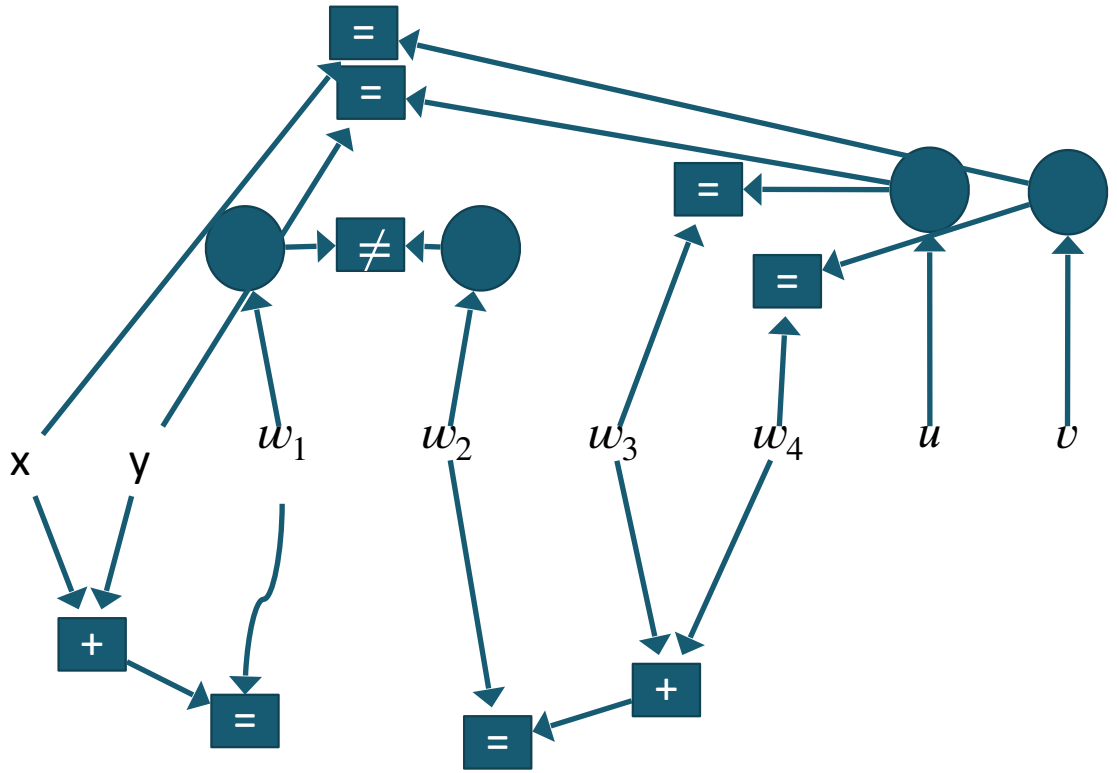
Uninterpreted Fun.

$$f(w_1) \neq f(w_2)$$
$$g(u) = y$$
$$g(v) = x$$
$$w_3 = g(u)$$
$$w_4 = g(v)$$
$$x = w_4$$
$$y = w_3$$

Linear Integer Arithmetic

$$w_1 = x + y w_2 = w_3 + w_4$$

$$x = w_4$$
$$y = w_3$$



Equality Propagation

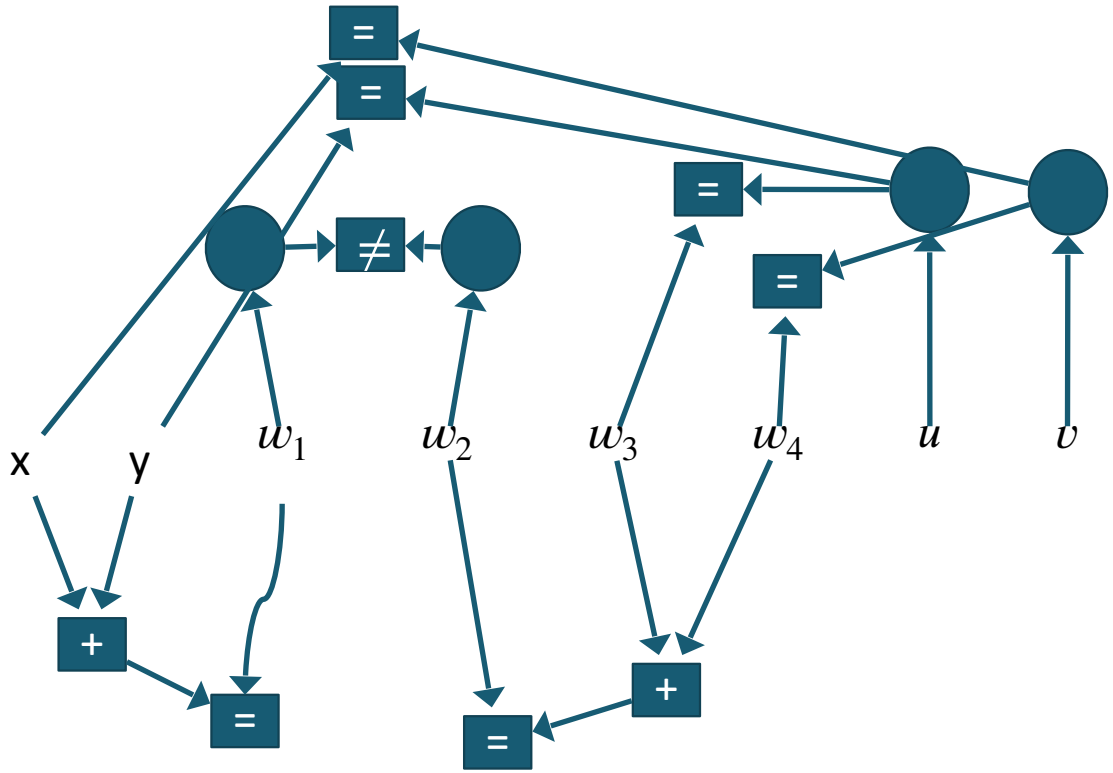
1) Run each UNSAT procedure independently
   - If any returns UNSAT we are done
2) Broadcast any equalities that were discovered
   - If no new equalities were discovered, return SAT

Uninterpreted Fun.

$f(w_1) \neq f(w_2)$
$g(u) = y$
$g(v) = x$
$w_3 = g(u)$
$w_4 = g(v)$
$x = w_4$
$y = w_3$
$w_1 = w_2$

Linear Integer Arithmetic

$w_1 = x + y w_2 = w_3 + w_4$
$x = w_4$
$y = w_3$
$w_1 = w_2$



Equality Propagation

1) Run each UNSAT procedure independently
   - If any returns UNSAT we are done
2) Broadcast any equalities that were discovered
   - If no new equalities were discovered, return SAT

Uninterpreted Fun.

$f(w_1) \neq f(w_2)$

$g(u) = y$

$g(v) = x$

$w_3 = g(u)$

$w_4 = g(v)$

$x = w_4$

$y = w_3$

$w_1 = w_2$

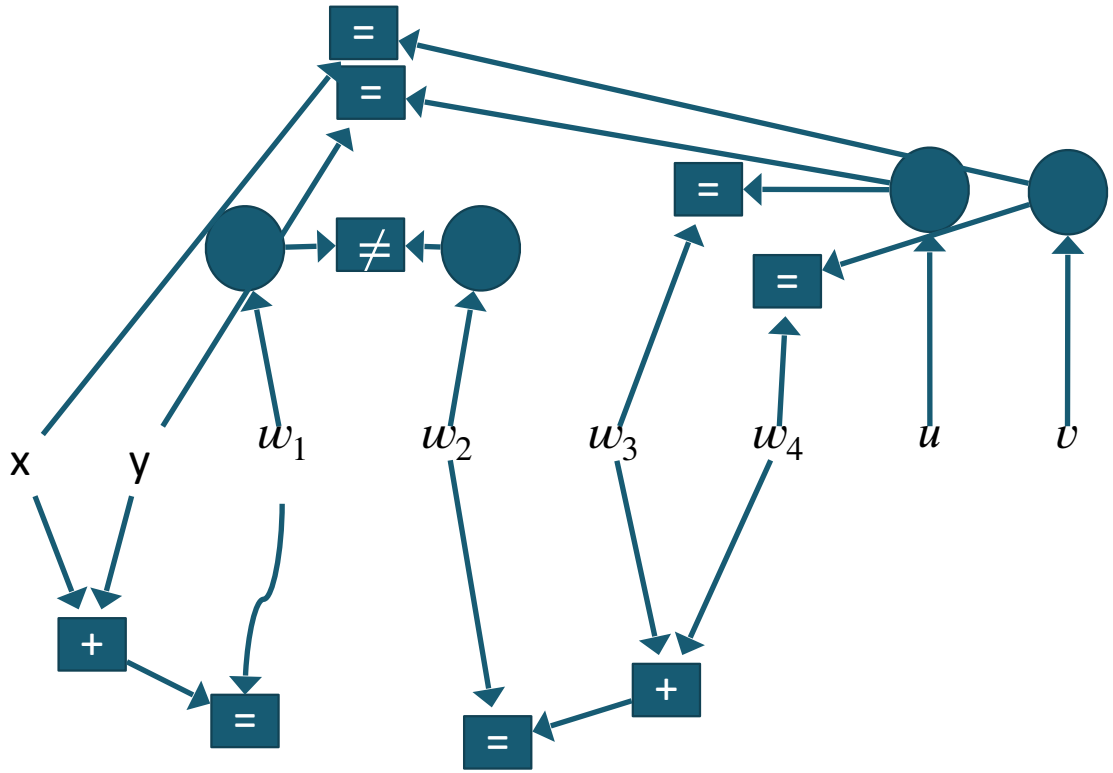
Linear Integer Arithmetic

$w_1 = x + yw_2 = w_3 + w_4$

$x = w_4$

$y = w_3$

$w_1 = w_2$



Soundness and Completeness

Soundness (when it says unsat, it is true)

- Yes assuming theory solvers don’t lie

Completeness (will it always say unsat when required?)

- Yes under certain conditions
- Argument not as obvious

Convex theories

Consider the formula:

$1 \leq x$

$x \leq 2$

$f(x) \neq f(1)$

$f(x) \neq f(2)$

Theory E

$1 \leq xx \leq 2w_1 = 1w_2 = 2$

Theory R

$f(x) \neq f\left(w_1\right) f(x) \neq f(w_2)$

- No equalities to propagate
- The two subformulas are satisfiable
- Yet, the problem is clearly unsat!

Convex theories

A theory is convex if for every conjunctive formula F

$$F \Rightarrow \bigvee_i x_i = y_i$$

$$implies$$

$$\exists i \;\; F \Rightarrow x_i = y_i$$

In the previous example:

- $1 \leq x \wedge x \leq 2 \Rightarrow (x = 1) \vee (x = 2)$
- It does not imply either $(x = 1) or (x = 2)$

Completeness

Nelson-Oppen is complete for Convex theories

What to do for non-convex ones?

Backtracking

Consider the formula:

$1 \leq x$

$x \leq 2$

$f(x) \neq f(1)$

$f(x) \neq f(2)$

ϕ_E in theory E

$1 \leq xx \leq 2w_1 = 1w_2 = 2$

ϕ_R in theory R

$f(x) \neq f\left(w_1\right) f(x) \neq f(w_2)$

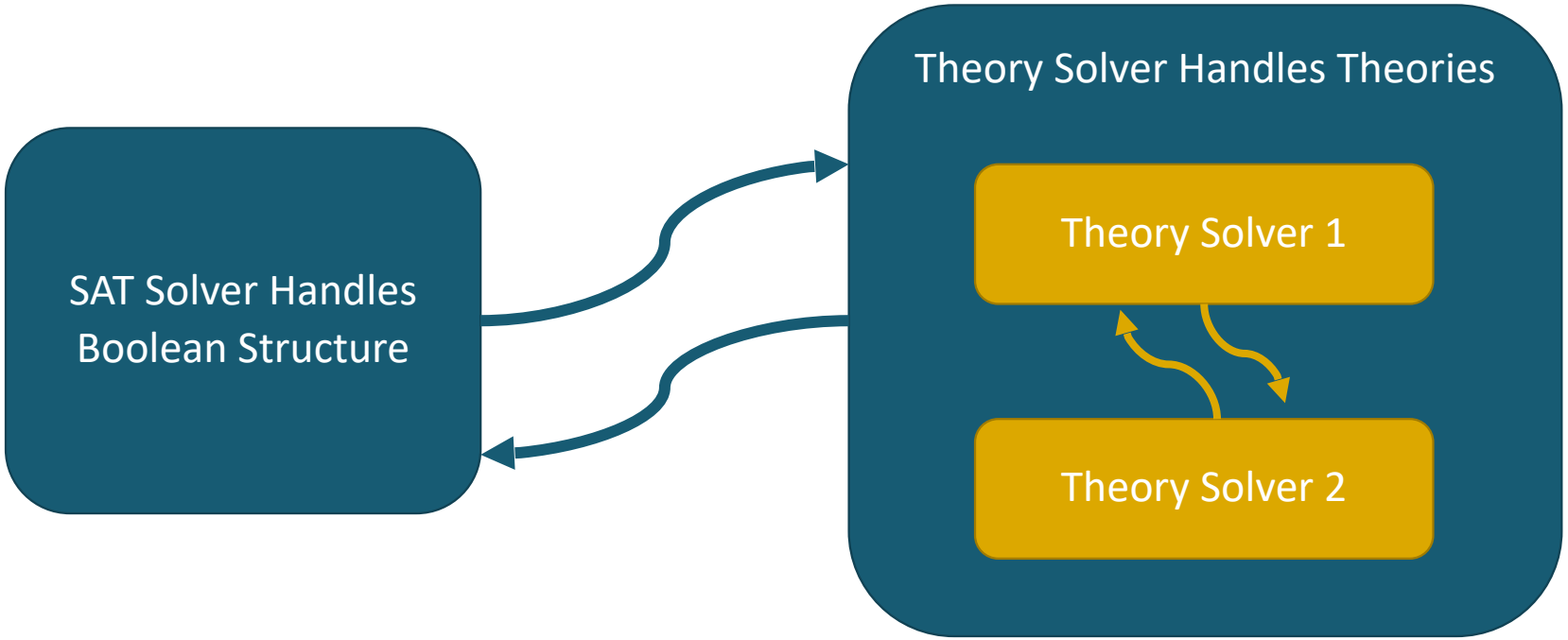
- Formula from theory E implies $(x = w_1) \vee (x = w_2)$.
- Try $\phi_R \wedge \left(x = w_1\right)$
- Try $\phi_R \wedge \left(x = w_2\right)$
- Since both are unsat, we can say UNSAT

Backtracking

Key issues

- How do we decide which disjunctive equalities to branch on?
- How to perform backtracking efficiently?

DPLL(T) Key idea



Boolean Abstraction

Given a formula F, a boolean abstraction B_F has the following property

- $unsat(B_F) \Rightarrow unsat(F)$

Eager approach

Simple, sometimes efficient

- Find a boolean abstraction s.t. $unsat(F) \Rightarrow unsat(B_F)$
- Give B_F to a sat solver
- DONE!

Can’t really pull it off if you have an infinite theory

Lazy approach

Start with a simple boolean abstraction

Check solutions against theory solver

If theory solver determines unsat improve your boolean abstraction

Repeat until either theory solver agrees or boolean abstraction becomes unsat

